Intelligent Agents

Chapter 2

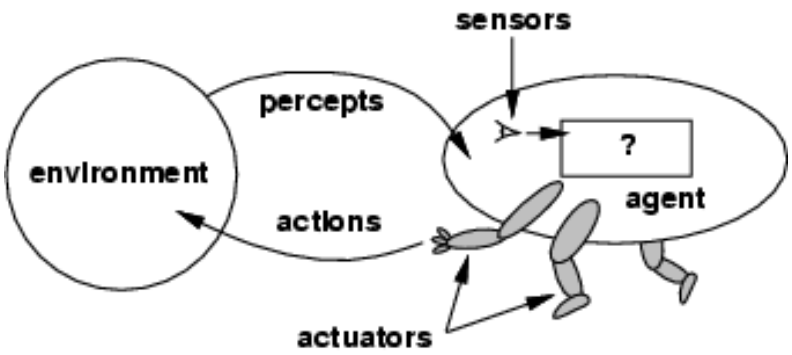
Outline

- Agents and environments
- Rationality
- PEAS (Performance measure, Environment, Actuators, Sensors)
- Environment types
- Agent types

Agents

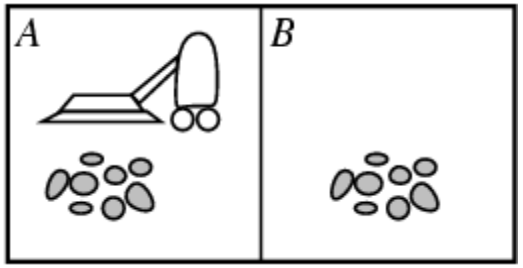
- An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators
-
- Human agent: eyes, ears, and other organs for sensors; hands,legs, mouth, and other body parts for actuators
- Robotic agent: cameras and infrared range finders for sensors;various motors for actuators

Agents and environments



- The agent function maps from percept histories to actions:
- $$[f: \mathcal{P}^{\star} \rightarrow \mathcal{A}]$$
- The agent program runs on the physical architecture to produce *f*
-

Vacuum-cleaner world



Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2. The agent cleans the current square if it is dirty, otherwise it moves to the other square. Note that the table is of unbounded size unless there is a restriction on the length of possible percept sequences.

- Percepts: location and contents, e.g., [A,Dirty]
- Actions: *Left*, *Right*, *Suck*, *NoOp*
-

A vacuum-cleaner agent

- \input{tables/vacuum-agent-function-table}
-

Rational agents

- An agent should strive to "do the right thing", based on what it can perceive and the actions it can perform. The right action is the one that will cause the agent to be most successful
-
- Performance measure: An objective criterion for success of an agent's behavior
-
- E.g., performance measure of a vacuum-cleaner agent could be amount of dirt cleaned up, amount of time taken, amount of electricity consumed, amount of noise generated, etc.
-

Rational agents

- Rational Agent: For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.
-

Rational agents

- Rationality is distinct from omniscience (all-knowing with infinite knowledge)
-
- Agents can perform actions in order to modify future percepts so as to obtain useful information (information gathering, exploration)
-
- An agent is autonomous if its behavior is determined by its own experience (with ability to learn and adapt)

PEAS

- PEAS: Performance measure, Environment, Actuators, Sensors
- Must first specify the setting for intelligent agent design
-
- Consider, e.g., the task of designing an automated taxi driver:
-
  - Performance measure
  -
  - Environment
  - Actuators
  - Sensors

PEAS

- Must first specify the setting for intelligent agent design
-
- Consider, e.g., the task of designing an automated taxi driver:
-
  - Performance measure: Safe, fast, legal, comfortable trip, maximize profits
  -
  - Environment: Roads, other traffic, pedestrians, customers
  -
  - Actuators: Steering wheel, accelerator, brake, signal, horn

PEAS

- Agent: Medical diagnosis system
- Performance measure: Healthy patient, minimize costs, lawsuits
- Environment: Patient, hospital, staff
- Actuators: Screen display (questions, tests, diagnoses, treatments, referrals)
-
- Sensors: Keyboard (entry of symptoms, findings, patient's answers)

PEAS

- Agent: Part-picking robot
- Performance measure: Percentage of parts in correct bins
- Environment: Conveyor belt with parts, bins
- Actuators: Jointed arm and hand
- Sensors: Camera, joint angle sensors

PEAS

- Agent: Interactive English tutor
- Performance measure: Maximize student's score on test
- Environment: Set of students
- Actuators: Screen display (exercises, suggestions, corrections)
- Sensors: Keyboard

Environment types

- Fully observable (vs. partially observable): An agent's sensors give it access to the complete state of the environment at each point in time.
-
- Deterministic (vs. stochastic): The next state of the environment is completely determined by the current state and the action executed by the agent. (If the environment is deterministic except for the actions of other agents, then the environment is strategic)
-
- Episodic (vs. sequential): The agent's experience is divided into atomic "episodes" (each episode consists of the agent perceiving and then performing a single action), and the choice of action in each episode depends only on the episode itself.

Environment types

- Static (vs. dynamic): The environment is unchanged while an agent is deliberating. (The environment is semidynamic if the environment itself does not change with the passage of time but the agent's performance score does)
-
- Discrete (vs. continuous): A limited number of distinct, clearly defined percepts and actions.
-
- Single agent (vs. multiagent): An agent operating by itself in an environment.

Environment types

	Chess with a clock	Chess without a clock	Taxi driving
Fully observable	Yes	Yes	No
Deterministic	Strategic	Strategic	No
Episodic	No	No	No
Static	Semi	Yes	No
Discrete	Yes	Yes	No
Single agent	No	No	No

- The environment type largely determines the agent design
-
- The real world is (of course) partially observable, stochastic, sequential, dynamic, continuous, multi-agent
-

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Figure 2.6 Examples of task environments and their characteristics.

Structure of Agent

Architecture + Program

The job of AI is to design an agent program that implements agent function - mapping percept to action. We assume that it will run on some sort of computing device with physical sensors and actuator.

Agent program depends only on current percept not percept history. If required history needs to be stored

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
              table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

Table-lookup agent

- \input{algorithms/table-agent-algorithm}
- Drawbacks:
  - Huge table even for chess – 10^150(no of atoms in observable universe is less than 10^80) – combinatorial explosion
  - Take a long time to build the table
  - No autonomy
  - Even with learning, need a long time to learn the table entries
  - Despite this it does it work assuming table is available

Agent types

- Four basic types in order of increasing generality:
- Simple reflex agents
- Model-based reflex agents
- Goal-based agents
- Utility-based agents

Simple reflex agents

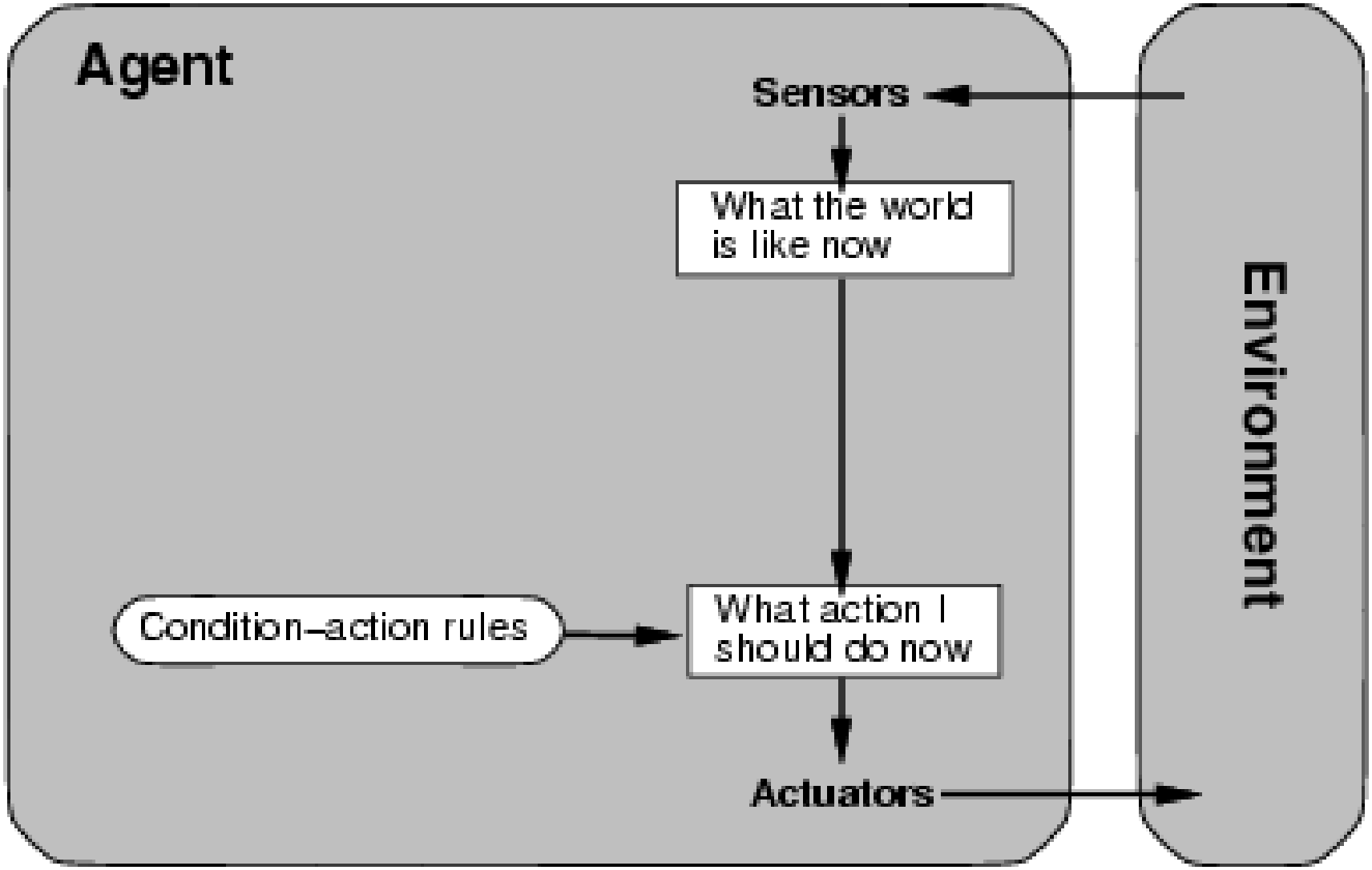


Figure 2.11 A model-based reflex agent.

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition–action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Repetition –
Infinite loop – not rational, too many condition inside loop
Randomized – may not be rational
Based on some input signal

Figure 2.10 A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.

Limitation of Simple Reflex Agent

- Action based on current percept
- Not possible if partially observable
- Not proper if mapping between percept and actual observation not proper

Model-based reflex agents

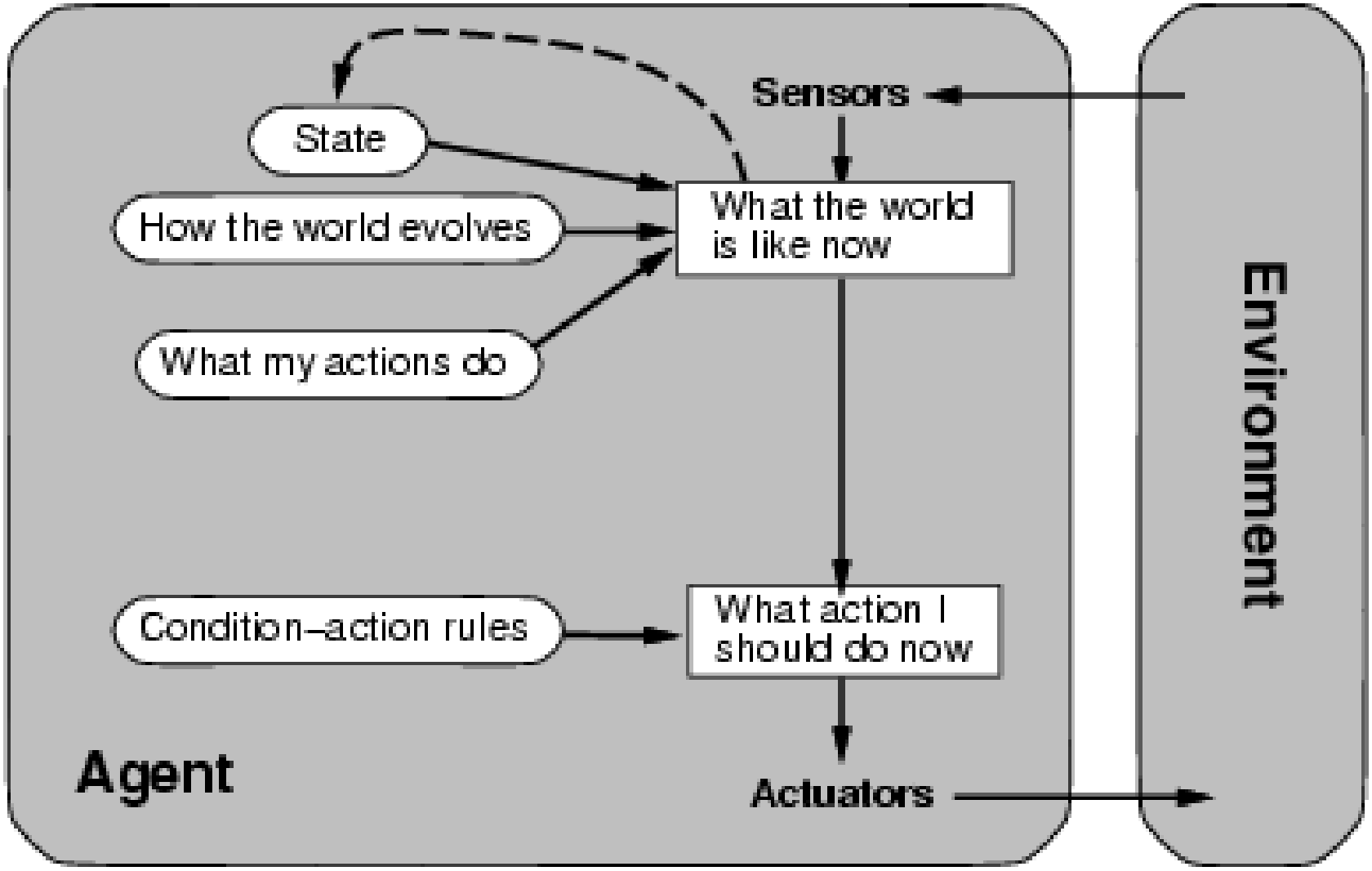


Figure 2.13 A model-based, goal-based agent. It keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.

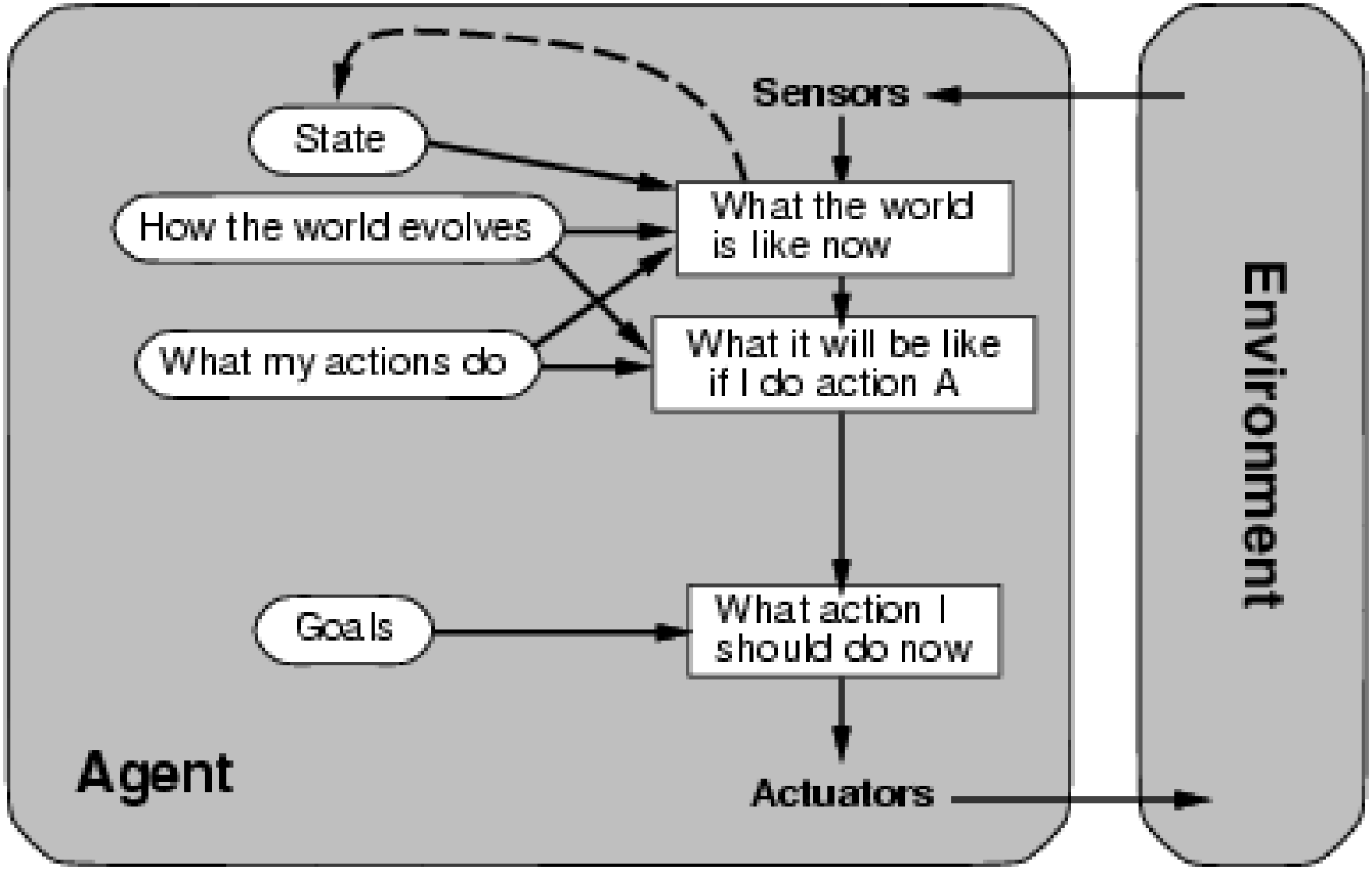
Model-Based Reflex Agent

- Keeps track of world it can not see maintaining an internal state that depends on precept history and reflects some unobserved aspects of current state.
- Car break ? Vaccum Cleaner ?
- Updating internal state is important in design . Requires two types of knowledge
  - Transition model –how world changes
  - Sensor model – how it is reflected through sensor
  - Transition + Sensor model = Model Based agent

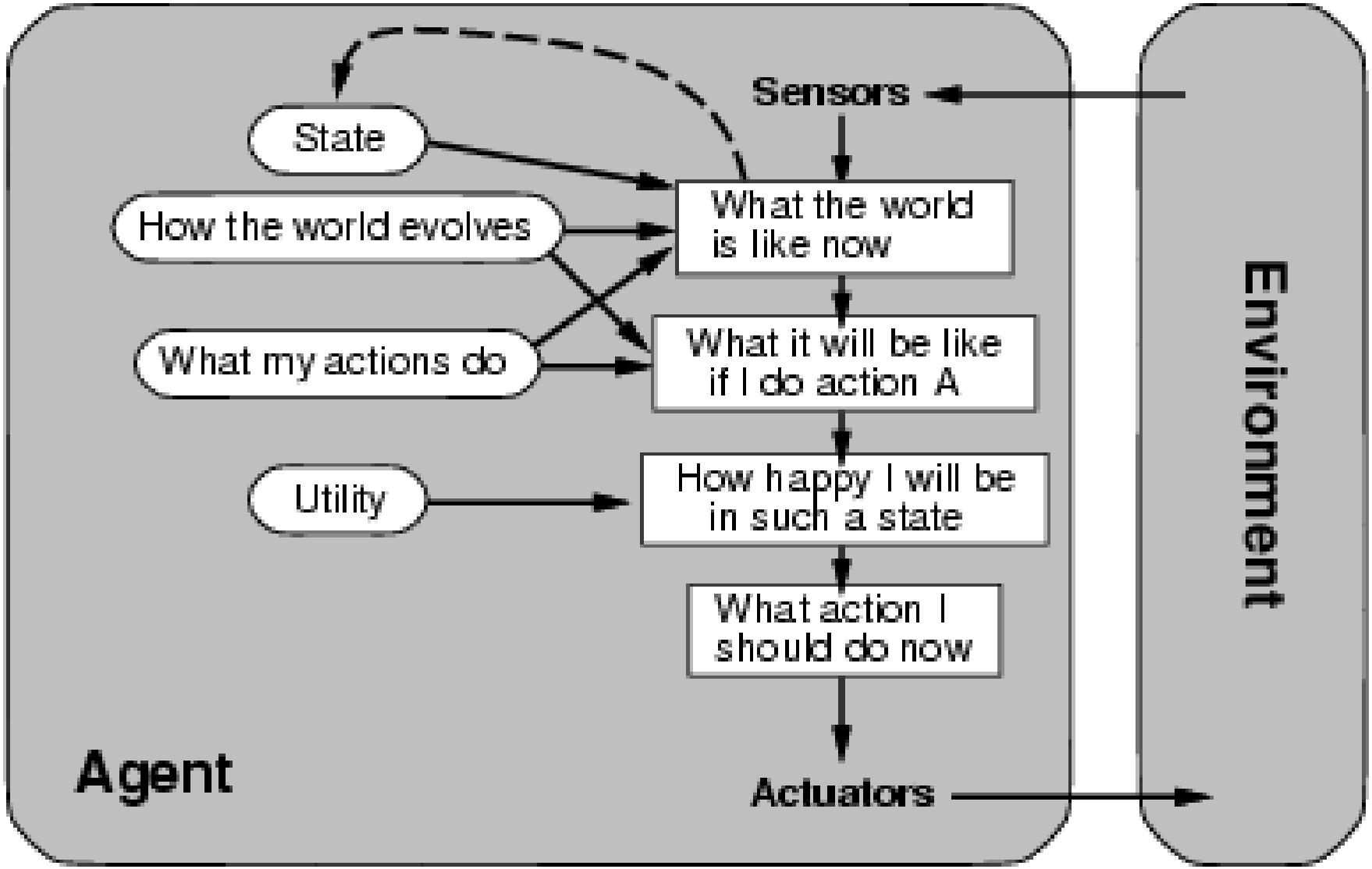
```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
              transition_model, a description of how the next state depends on
                    the current state and action
              sensor_model, a description of how the current world state is reflected
                    in the agent's percepts
              rules, a set of condition–action rules
              action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, transition_model, sensor_model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Figure 2.12 A model-based reflex agent. It keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

Goal-based agents



Utility-based agents



Learning agents